

2802420681 *Chris William Kurniawan*

2802392671 *Michael Roland*

2802394670 *Revan Marfrando Cokro*

1. PROJECT ABSTRACT & DESCRIPTION

Genshin Import is a state-of-the-art mobile e-commerce hybrid application tailored around the rich visual fantasy aesthetics of *Genshin Impact*. The platform enables users (Travelers) to browse and buy legendary Teyvat weapons and artifacts, while allowing administrators (Store Owners) to manage inventory parameters directly via high-fidelity, validated UI dashboards.

The project incorporates:

- A high-performance relational MySQL database for persistent transactions.
- A secure Node.js + Express.js backend executing robust bearer-authentication guards and OAuth workflows.
- A responsive, fluid Flutter mobile client deploying beautiful animations, structured inputs, and cohesive material widgets.

2. THEMATIC UI DESIGN & CREATIVITY

To fulfill the project theme criteria, Genshin Import is customized with a Premium Dark Mystical & Gold Theme inspired directly by the in-game user interfaces of high-fantasy RPGs.

Customized Global UI Properties:

1. Typography (Font Family):

- GoogleFonts.cinzel: Used for all major page headers, dialog titles, and purchase receipts to provide an authentic, ancient RPG armory feel.
- GoogleFonts.inter: Used for product descriptions, form fields, and system logs to maintain optimal textual readability across small mobile screens.

2. Color Scheme & Sufficiency Contrast:

- Primary/Tint Color: Primogem Gold (0xFFD3BC8E) and Antique Muted Gold (0xFFA78B50) are utilized for highlights, active tabs, buttons, and borders.
- Background Canvas: Obsidian Dark Sky (0xFF0A0F1D) with Deep Slate Cards (0xFF161F32), satisfying high contrast guidelines to prevent user fatigue or visual clashing.
- Accent Highlights: Soft Elemental Cyan (0xFF3AAFA9) for notifications/badges and Pyro Red (0xFFE06C75) for warning/delete triggers.

3. DATABASE SCHEMATIC (MYSQL)

The database structure is designed to isolate security concerns, establish role-based access, and support full relational operations.

Table: roles

Stores authorization groups.

- id (INT AUTO_INCREMENT, PRIMARY KEY): Unique role identifier.
- name (VARCHAR(50), UNIQUE): Access group name (Admin, User).

Table: users

Stores user profile credentials and role relations.

- id (INT AUTO_INCREMENT, PRIMARY KEY): Unique traveler identifier.
- name (VARCHAR(100)): Display name of the user.
- email (VARCHAR(100), UNIQUE): Login username.
- password (VARCHAR(255)): Secure password stored via cryptographic hashes (bcryptjs).
- role_id (INT): References roles(id) (Foreign Key).

Table: items

Stores Weapons and Artifacts inventory parameters.

- id (INT AUTO_INCREMENT, PRIMARY KEY): Teyvat serial identifier.
- name (VARCHAR(150)): Item name.
- type (VARCHAR(50)): Classification category (Weapon, Artifact).
- description (TEXT): Weapon/Artifact background lore description.
- stock (INT): Quantity available in storage
- image (VARCHAR(255)): URL link to the visual graphic asset.
- price (DECIMAL(10,2)): Purchasing Mora cost

4. BACK-END API SPECIFICATIONS

API Endpoint Registry

Method	Endpoint	Authorization	Description	Input Params
POST	/api/auth/login	Public	Validates user password against DB and returns session Bearer Token.	{email, password}
POST	/api/auth/oauth	Public	auto-registering new OAuth emails.	{provider, email, name}
GET	/api/items	Public	Retrieves all Weapons and Artifacts from store inventory.	None
GET	/api/items/:id	Public	Retrieves detailed information of a single item.	:id (Path variable)
POST	/api/items	Bearer Token (Admin)	Inserts a new item into inventory. Stock/price must not be negative.	{name, type, description, stock, image, price}
PUT	/api/items/:id	Bearer Token (Admin)	Updates parameters of an existing item in inventory.	{name, type, description, stock, image, price}
DELETE	/api/items/:id	Bearer Token (Admin)	Deletes an item permanently from inventory database.	:id (Path variable)
POST	/api/items/buy	Bearer Token (User)	Subtracts selected quantities from stock. Validates stock bounds.	{cart: [{id, quantity}]}

5. SECURITY ARCHITECTURE & OAUTH

1. Password Hashing: Stored user passwords are encrypted using bcryptjs with a work factor of 10 salt rounds to secure credentials in the MySQL schema.
2. Bearer Token Generation: Upon validation, the server generates a cryptographically random, alphanumeric session Bearer Token (24 characters) exceeding the required 20-character minimum (e.g., n8x7wfqtsrvxnvsm8dcz).
3. Token Guard Middleware: The /api/items endpoints are protected by verifyToken middleware, parsing the incoming Authorization header (Bearer token). Non-admin requests attempting write actions are blocked with a 403 Forbidden error.
4. External OAuth Flow (Google): Standardized OAuth automatically handles profile matching. In cases where the user logs in via external OAuth for the first time, the server generates a randomized secure password, inserts the user row as a standard traveler, and returns a session Bearer Token instantly.

6. FRONT-END FLUTTER PAGES TOUR

Page 1: Login Page (login_page.dart)

- Visual Elements: Dynamic background gradient, a mystic floating star icon, credentials card, and distinct buttons for Google login.
- Data **Validations:**
 - Email Regex: Enforces valid format
 - Required Fields: Blocks submissions containing empty inputs.
 - Length Check: Enforces a minimum password length of 4 characters.

Page 2: Dashboard / Home Page (dashboard_page.dart)

- Visual Elements: A responsive grid containing items styled as premium 5-star RPG cards with custom rating stars. Displays dynamic badges for item type (Weapon/Artifact) and stock.
- Interactivity: Category selection tabs (All, Weapons, Artifacts), realtime text search filter, user role banner displays, and a dynamic shopping cart counter.
- Admin Access: Admin users are presented with a persistent "MANAGE ITEMS" floating action button that opens the administration dashboard.

Page 3: Detail Product Page (detail_page.dart)

- Visual Elements: Fading image banner overlay, serial ID fields, and immersive background lore descriptions.
- Interactivity: Integrates an elegant Quantity Selector allowing users to adjust order size. Shows a real-time subtotal calculation.
- **Validations:** The "Add to Cart" button is automatically disabled if stock is 0. Selected quantity is validated to stay within the item's maximum stock limit.

Page 4: Admin Management Page (admin_page.dart)

- Visual Elements: Row-based list of the entire inventory showing thumbnails, prices, types, and stock. Action buttons for editing (pencil) and deleting (trash bin) items are available.
- Validated Form Dialogue: Floating form fields for adding or editing items.
- **Data Validations:**
 - Fields cannot be empty.
 - Stock inputs cannot be negative.
 - Price inputs cannot be negative.
 - Form matches rules: name must 3 characters, and description must be 5 characters.
 - If validation fails, display accurate error details.

Page 5: Cart / Transaction Summary Page (cart_page.dart)

- Visual Elements: List of added items showing item images, individual pricing, and cost subtotals.
- Financial Invoice Receipt: Displays a detailed transaction invoice
- Checkout & Invoice Summary: Clicking the "Confirm Checkout" button hits the backend /buy API. Successful checkouts display an ancient "Adventurer's Receipt" modal invoice showing purchase details, and clear the local cart state.

STEP-BY-STEP USER SETUP INSTRUCTIONS

Step 1: MySQL Database Setup & Importing SQL Schema

Before launching the servers, you must set up the database and import the relational schema.

Option A: Web Import via phpMyAdmin

1. Open the XAMPP Control Panel on your system.
2. Click Start next to the Apache and MySQL modules.
3. Open your web browser and navigate to the graphical database controller page:
<http://localhost/phpmyadmin>.
4. Click on the Import tab located on the top navigation bar.
5. Click the Choose File button, navigate inside the project's backend/ directory, and select the file database.sql.
6. Scroll down to the bottom of the import window and click the Go (or Import) button.
7. The database named genshin_import, all tables (roles, users, items), and all pre-seeded travel entries will be automatically created.

Step 2: Boot Node.js Backend Server

1. Open your terminal inside the backend/ directory.
2. Install dependencies:

```
npm install
```

3. Boot the server:

```
npm run start
```

Log output will display connection status to MySQL. If MySQL in XAMPP is active, it connects instantly. Otherwise, it triggers the automatic In-Memory database.

Step 3: Boot Flutter Client Application

1. Open your terminal inside the frontend/ directory.
2. Fetch packages:

```
flutter pub get
```

3. Launch the app:

```
flutter run
```

1. Akun Admin (Store Owner)

- Email: admin@genshin.com
- Password: admin123

2. Akun Traveler (User Biasa)

- Email: user@genshin.com
- Password: user123

Reference

1. Blade of Despair (Item Kategori: Weapon)

https://mobile-legends.fandom.com/wiki/Blade_of_Despair

2. Dominance Ice (Item Kategori: Artifact/Equipment)

https://mobile-legends.fandom.com/wiki/Dominance_Ice

3. Rose Gold Meteor (Item Kategori: Weapon)

https://mobile-legends.fandom.com/wiki/Rose_Gold_Meteor

4. Windtalker (Item Kategori: Weapon)

<https://mobile-legends.fandom.com/wiki/Windtalker>

5. Sea Halberd (Item Kategori: Weapon)

https://mobile-legends.fandom.com/wiki/Sea_Halberd